

LAPORAN TUGAS PRAKTIKUM
STRUKTUR DATA
PERTEMUAN 10



Nama : Jaw Syanito Riptanto Putra
NPM : 2510631170040
Kelas : 2 A

PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG

2025

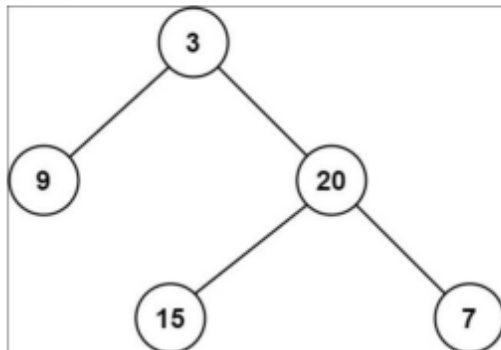
SOAL

Latihan

Temukan batas kedalaman dari sebuah binary tree.

Batas kedalaman adalah jumlah node jalur terpendek dari akar hingga ke daun. Node daun adalah node yang tidak memiliki turunan (no child)

Contoh



Input : 3,9,20,null,null,15,7

Output : 2

JAWABAN

Penjelasan :

Saya membuat program berbasis C++ yang berfungsi untuk menghitung batas kedalaman minimum (minimum depth) pada sebuah Binary Tree menggunakan metode rekursi. Program ini menggunakan struktur data tree yang terdiri dari beberapa node yang saling terhubung antara parent dan child. Tujuan dari program ini adalah untuk memahami konsep struktur data Binary Tree, penggunaan struct, pointer, fungsi rekursif, percabangan, serta proses penelusuran tree untuk mencari jalur terdangkal dari root menuju leaf node.

Di bagian awal program, saya menambahkan `#include <iostream>` agar program dapat menggunakan fungsi input dan output seperti `cout`. Kemudian saya menggunakan `using namespace std;` supaya penulisan fungsi standar C++ menjadi lebih sederhana tanpa perlu menggunakan awalan `std::`.

Selanjutnya saya membuat sebuah struct bernama `TreeNode` yang digunakan untuk membentuk node pada Binary Tree. Di dalam struct tersebut terdapat variabel nilai bertipe `int` yang digunakan untuk menyimpan data angka pada node. Selain itu terdapat dua pointer yaitu `left` dan `right`. Pointer `left` digunakan untuk menunjuk child sebelah kiri, sedangkan pointer `right` digunakan untuk menunjuk child sebelah kanan dari sebuah node.

Di dalam struct `TreeNode`, saya juga membuat constructor `TreeNode(int x)` yang berfungsi untuk menginisialisasi node baru ketika dibuat. Nilai parameter `x` akan disimpan ke dalam variabel nilai. Kemudian pointer `left` dan `right` diatur menjadi `NULL` yang menandakan bahwa node tersebut belum memiliki child.

Setelah membuat struct node, saya membuat sebuah fungsi bernama `minimum(int a, int b)` yang digunakan untuk mencari nilai terkecil dari dua buah angka. Fungsi ini menggunakan percabangan `if` untuk membandingkan nilai `a` dan `b`. Jika nilai `a` lebih kecil dari `b`, maka fungsi akan mengembalikan nilai `a`. Jika tidak, maka fungsi akan mengembalikan nilai `b`.

Selanjutnya saya membuat fungsi rekursif bernama `minKedalaman(TreeNode* root)` yang digunakan untuk menghitung kedalaman minimum pada Binary Tree. Fungsi ini menerima parameter berupa pointer `root` yang menunjuk node awal tree.

Pada bagian awal fungsi, program melakukan pengecekan menggunakan kondisi `if (root == NULL)`. Kondisi ini digunakan untuk memeriksa apakah tree kosong atau tidak memiliki node. Jika kondisi terpenuhi, maka fungsi akan mengembalikan nilai `0` karena tidak ada kedalaman pada tree yang kosong.

Setelah itu program memeriksa apakah node saat ini merupakan leaf node menggunakan kondisi `if (root->left == NULL && root->right == NULL)`. Leaf node adalah node yang tidak memiliki child kiri maupun kanan. Jika kondisi ini terpenuhi, maka fungsi mengembalikan nilai `1` karena node tersebut merupakan akhir dari jalur tree.

Kemudian program melakukan pengecekan apakah child kiri kosong menggunakan kondisi `if (root->left == NULL)`. Jika child kiri tidak ada, maka program hanya akan melanjutkan pencarian kedalaman melalui child kanan dengan memanggil fungsi `minKedalaman(root->right) + 1`. Penambahan angka `1` digunakan untuk menghitung node saat ini sebagai bagian dari kedalaman.

Selanjutnya program juga memeriksa apakah child kanan kosong menggunakan kondisi `if (root->right == NULL)`. Jika child kanan tidak ada, maka program melanjutkan pencarian kedalaman melalui child kiri menggunakan `minKedalaman(root->left) + 1`.

Jika kedua child tersedia, maka program akan mencari kedalaman terkecil dari subtree kiri dan subtree kanan. Hal ini dilakukan menggunakan fungsi `minimum(minKedalaman(root->left), minKedalaman(root->right)) + 1`. Fungsi rekursif akan terus berjalan sampai mencapai leaf node, kemudian hasil kedalaman terkecil dikembalikan secara bertahap ke pemanggil sebelumnya.

Di dalam fungsi utama `main()`, saya membuat Binary Tree secara manual menggunakan dynamic memory allocation dengan keyword `new`. Pertama saya membuat root node dengan nilai 3 menggunakan `TreeNode* root = new TreeNode(3);`.

Selanjutnya saya membuat child kiri dari root dengan nilai 9 menggunakan `root->left = new TreeNode(9);`. Setelah itu saya membuat child kanan dari root dengan nilai 20 menggunakan `root->right = new TreeNode(20);`.

Kemudian node bernilai 20 diberikan dua child lagi yaitu node 15 sebagai child kiri dan node 7 sebagai child kanan. Dengan cara ini terbentuk sebuah Binary Tree yang memiliki beberapa jalur kedalaman berbeda.

Setelah tree selesai dibuat, program menampilkan hasil kedalaman minimum menggunakan perintah `cout << "Batas Kedalaman : " << minKedalaman(root);`. Fungsi `minKedalaman(root)` akan menghitung jalur terdangkal dari root menuju leaf node, kemudian hasilnya ditampilkan ke layar.

Terakhir program menggunakan `return 0;` untuk menandakan bahwa program telah selesai dijalankan tanpa error.

Dengan cara tersebut, program dapat menghitung batas kedalaman minimum pada Binary Tree menggunakan metode rekursi. Program ini menunjukkan penggunaan beberapa konsep penting dalam pemrograman C++, seperti `struct` untuk membuat node tree, pointer untuk menghubungkan node, dynamic memory allocation menggunakan `new`, percabangan untuk memeriksa kondisi node, serta rekursi untuk melakukan penelusuran tree secara otomatis hingga mencapai leaf node.

Kode Programnya :

```
#include <iostream>
using namespace std;

// Struktur node binary tree
struct TreeNode {
    int nilai;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) {
        nilai = x;
        left = NULL;
        right = NULL;
    }
};

// Fungsi mencari nilai minimum
int minimum(int a, int b) {
    if (a < b) {
        return a;
    }
    return b;
}

// Fungsi mencari kedalaman pakai rekursi
int minKedalaman(TreeNode* root) {
    // Jika tree kosong
    if (root == NULL) {
        return 0;
    }

    // Jika node daun
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }

    // Jika child kiri kosong
    if (root->left == NULL) {
        return minKedalaman(root->right) + 1;
    }

    // Jika child kanan kosong
    if (root->right == NULL) {
        return minKedalaman(root->left) + 1;
    }
}
```

```

    // Ambil depth terkecil
    return minimum(minKedalaman(root->left), minKedalaman(root->right)) + 1;
}

int main() {

    // Membuat binary tree
    TreeNode* root = new TreeNode(3);

    root->left = new TreeNode(9);

    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);
    root->right->right = new TreeNode(7);

    // Menampilkan hasil
    cout << "Batas Kedalaman : " << minKedalaman(root);

    return 0;
}

```

Outputnya di terminal :

```
Batas Kedalaman : 2
```

Link Repository GITHUB :

https://github.com/jawsyanito/StrukturData_25040/tree/ffcdded305b4d1cd1c454bf9d9857c2219593d166/Pertemuan%2010/Tugas